# Pantry Pal (working title) — MVP Spec

> Hand this file to Claude Code at the repo root. Build in the order given in **Build Plan**. Ask before deviating from the stack or scope.

## 1. Problem & Goal

Food gets forgotten in the fridge and goes bad. The app should make it **nearly effortless to log what I buy**, **tell me what to eat before it expires**, and **help me plan meals around what I already have**.

**North-star metric:** items thrown away expired / items added (lower is better).

**Design principle:** every feature must be reachable in ⩽2 taps from the home screen, and adding an item must take ⩽5 seconds on average. If a feature adds friction to logging, cut it.

## 2. Users & Assumptions

- Single user (me). No sharing, roles, or multi-household logic. Still scope all data by `userId` so it can expand later.
- Used mostly on a phone, often in a kitchen or grocery store, one-handed.
- "Tracking food" = tracking **inventory and its freshness** (what I have, where, when it expires). Calorie/macro tracking is **out of scope** for MVP (but store nutrition data from barcode lookups if available, for later).

## 3. Platform & Stack

| Concern | Choice |
|---|---|
| App type | Installable **PWA** (mobile-first, works offline for read/add) |
| Framework | Next.js (App Router) + TypeScript |
| Styling | Tailwind CSS (+ shadcn/ui for primitives) |
| Auth + DB | Firebase Auth (Google sign-in) + Firestore (offline persistence on) |
| File storage | Firebase Storage (receipt images, deleted after parsing) |
| AI | Anthropic API, called **server-side only** via Next.js route handlers (receipt vision, text parsing, expiry estimates, meal suggestions) |
| Barcode lookup | Open Food Facts API (free, no key) |
| Barcode scanning | `BarcodeDetector` API where supported, fallback to `@zxing/browser` (iOS Safari lacks BarcodeDetector) |
| Notifications | Web Push (works on iOS 16.4+ only when installed to Home Screen) |
| Hosting | Vercel |

Secrets (Anthropic key, Firebase admin creds) live in env vars. Never expose them to the client.

## 4. Core Features (MVP)

### 4.1 Add Food — the most important feature

A single persistent **"+" button** opens an Add sheet with four entry methods. All methods feed into the same **Review & Confirm** screen so the user can fix mistakes in one place.

1. **Quick type / voice**
   - One text box: "milk, dozen eggs, 2 chicken breasts, bag of spinach".
   - Mic button uses the Web Speech API where available; otherwise the phone keyboard's dictation works in the same text box.
   - Server sends text to Claude → returns structured items (name, quantity, unit, category, storage location, estimated expiry).
2. **Barcode scan**
   - Camera view, continuous scan. On match, look up Open Food Facts → name, brand, category, image, nutrition (store it).
   - Scanning multiple items in a row adds each to a pending list (don't reopen the camera each time).
   - If the barcode isn't found, fall back to a manual name field with the barcode saved, and remember it for next time (`barcodeCache` collection).
3. **Receipt photo**
   - Take or upload a photo (allow multiple photos for long receipts).
   - Server sends the image to Claude vision → returns a list of **food items only** (ignore tax, bags, non-food), with cleaned-up names (e.g. `ORG BNLS SKNLS CHKN BRST` → "Chicken breast"), quantities, and estimates.
   - Delete the image from storage once parsing succeeds.
4. **Manual form** (fallback only): name, qty, unit, location, expiry date.

**Review & Confirm screen**

- List of parsed items, each editable inline (name, qty, location, expiry).
- Swipe to remove a wrong item. "Add all" is a single big button.
- Show a small "estimated" badge on any AI-guessed expiry date.

**4.2 Inventory (Home screen)**

- Default view: items **sorted by expiry soonest first**, grouped by status: **Expired** / **Use today** / **This week** / **Later**.
- Color-coded freshness (red/amber/green) with text labels, not color alone.

- Filter chips: All · Fridge · Freezer · Pantry.
- Search box.
- Tap an item → detail sheet with quick actions:
  - **Used it** (decrement quantity or remove)
  - **Threw it out** (logs as waste, with optional reason: expired / spoiled early / didn't like / leftovers)
  - **Move to freezer** (auto-extends expiry using a freezer shelf-life estimate)
  - Edit
- Swipe gestures on list rows: right = Used, left = Threw out.

### 4.3 Expiry Intelligence

- When an item is added without a date, estimate expiry from `category + storageLocation` using:
  1. A bundled default shelf-life table (`/lib/shelfLife.ts`) for common categories (fast, offline, deterministic).
  2. Claude estimate for anything not in the table.
- Mark source: `expirySource: "user" | "package" | "table" | "ai"`.
- Opened/cooked state is out of scope for MVP, but keep a nullable `openedAt` field in the schema.

### 4.4 Use-It-Up Alerts

- Daily push notification at a user-set time (default 5:00 PM): "3 items expire in the next 2 days: spinach, chicken breast, yogurt." Tapping opens the "Use soon" list.
- In-app banner on Home if anything expires within 2 days or is already expired.
- Fallback if push isn't permitted: the in-app banner only (don't block on push).
- Settings: notification time, "warn me N days before" (default 2).

### 4.5 Meal Planning

- **Weekly planner:** 7 days × (breakfast / lunch / dinner / snack) grid. Each slot holds a meal (free-text title + optional ingredient list).
- **"What should I cook?"** button:

    - Sends the current inventory (prioritizing items expiring soonest) to Claude.
    - Returns 3–5 meal ideas that use expiring items first, each with: title, ingredients (marking which are already in inventory vs. missing), short steps, estimated time.
    - One tap to **add to a planner slot** or **save as a recipe**.
- **Mark as cooked:** pick which inventory items were used; decrements them automatically.
- **Auto shopping list:** missing ingredients from planned meals roll into a simple checklist. Checking an item off offers "Add to inventory".
- Saved recipes: title, ingredients, steps, notes. Simple list, no import/scraping in MVP.

### 4.6 Waste Tracking (lightweight)

- Every "Threw it out" creates a `wasteLog` entry.
- Stats screen (simple): items wasted this week/month, most-wasted categories, waste rate = wasted ÷ added. A small bar chart is enough.

## 5. Data Model (Firestore)

All documents live under `users/{userId}/...`.

ts

```
// inventoryItems
{
  id: string,
  name: string,                 // normalized, human-readable
  category: string,             // "dairy" | "produce" | "meat" | "pantry" | ...
  quantity: number,
  unit: string,                 // "item" | "g" | "oz" | "lb" | "L" | "pack" ...
  location: "fridge" | "freezer" | "pantry",
  addedAt: Timestamp,
  expiresAt: Timestamp | null,
  expirySource: "user" | "package" | "table" | "ai",
  openedAt: Timestamp | null,   // reserved
  barcode: string | null,
  imageUrl: string | null,
  nutrition: object | null,     // raw from Open Food Facts, reserved for later
  status: "active" | "used" | "wasted"
}

// wasteLog
{ id, itemName, category, quantity, unit, reason, wastedAt, addedAt }

// mealPlan  (one doc per date: "YYYY-MM-DD")
{ date, slots: { breakfast?: Meal, lunch?: Meal, dinner?: Meal, snack?: Meal } }
// Meal: { title, recipeId?: string, ingredients: {name, qty?, unit?}[], cooked: boolean }

// recipes
{ id, title, ingredients: [...], steps: string[], timeMinutes?: number, notes?: string }

// shoppingList
{ id, name, qty?, unit?, checked: boolean, fromMealDate?: string }
```

```
// barcodeCache
{ barcode, name, category, brand?, imageUrl?, nutrition? }

// settings (single doc)
{ notifyTime: "17:00", warnDaysBefore: 2, pushSubscription?: object, timezone: string }
```

Add Firestore security rules: a user can only read/write documents under their own `users/{uid}`.

### 6. API Routes (Next.js route handlers, server-only AI calls)

| Route | Input | Output |
|---|---|---|
| `POST /api/parse-text` | `{ text }` | `{ items: ParsedItem[] }` |
| `POST /api/parse-receipt` | `{ images: base64[] }` | `{ items: ParsedItem[] }` |
| `GET /api/barcode/[code]` | — | `ParsedItem \| null` (checks `barcodeCache` first, then Open Food Facts) |
| `POST /api/suggest-meals` | `{ inventory, preferences? }` | `{ meals: MealSuggestion[] }` |
| `POST /api/estimate-expiry` | `{ name, category, location }` | `{ days: number }` |
| `POST /api/cron/daily-digest` | (Vercel Cron) | sends push notification |

Requirements for all AI routes:

- Verify the Firebase ID token on every request.

- Force **JSON-only** output from Claude with a strict schema; validate with Zod; retry once on parse failure.
- Rate-limit per user (simple in-memory or Firestore counter is fine for MVP).
- Use a cheaper/faster Claude model for parsing and estimates; keep the model name in one config file.

```ts
type ParsedItem = {
  name: string; quantity: number; unit: string;
  category: string; location: "fridge" | "freezer" | "pantry";
  estimatedShelfLifeDays: number | null; confidence: "high" | "medium" | "low";
  barcode?: string; imageUrl?: string;
};
```

## 7. Screens

1. **Home / Inventory** (default) — banner, grouped list, filters, search, floating "+".
2. **Add sheet** → Type/Voice · Scan · Receipt · Manual → **Review & Confirm**.
3. **Item detail sheet** — quick actions.
4. **Meal Plan** — weekly grid + "What should I cook?".
5. **Shopping List** — checklist.
6. **Recipes** — saved list + detail.
7. **Stats** — waste summary.
8. **Settings** — notifications, sign out.

Bottom tab bar: **Pantry · Plan · Shop · Stats**. Settings and Recipes live under a menu/header icon.

## 8. UX Requirements

- Mobile-first; thumb-reachable controls; minimum 44px tap targets.
- Installable (manifest, icons, service worker), works offline for viewing inventory and adding items (sync when back online).

- Dark mode following the system setting.
- Loading states for AI calls with skeletons; never block the UI on parsing—show “Reading your receipt…” and let me keep navigating.
- Every destructive action is undoable with a toast (“Removed. Undo”).
- Empty states with a clear next action.

## 9. Non-Goals (explicitly NOT in MVP)

- Calorie/macro tracking, nutrition goals
- Multi-user / household sharing
- Grocery store integrations or online ordering
- Recipe URL import
- Photo-of-fridge recognition
- Native iOS/Android apps
- Price tracking / budgeting

## 10. Acceptance Criteria

- [ ] I can add 10 groceries from a receipt photo and confirm them in under 60 seconds.
- [ ] Scanning 5 barcodes in a row adds 5 items without reopening the camera.
- [ ] Typing “milk, eggs, spinach” creates 3 items with sensible locations and expiry dates.
- [ ] Home screen clearly shows what to use first, sorted by expiry.
- [ ] I receive a daily push on my installed PWA listing items expiring soon.
- [ ] “What should I cook?” returns meals that prioritize expiring items and flags missing ingredients.
- [ ] Marking a meal cooked decrements the right inventory items.
- [ ] Logging an item as thrown out appears in Stats.
- [ ] App is installable and loads inventory offline.
- [ ] No API keys appear in the client bundle.

## 11. Build Plan (suggested order for Claude Code)

1. **Scaffold:** Next.js + TS + Tailwind + shadcn, PWA manifest/service worker, Firebase Auth (Google), Firestore rules, env setup.
2. **Inventory core:** data model, Home list with grouping/sorting, item detail actions, manual add, offline persistence.
3. **Expiry engine:** shelf-life table + `/api/estimate-expiry` + freshness UI.
4. **Quick type/voice add:** `/api/parse-text` + Review & Confirm screen (this screen is reused by everything below).
5. **Barcode scanning:** scanner component with fallback, Open Food Facts lookup, barcode cache, multi-scan flow.
6. **Receipt scanning:** `/api/parse-receipt`, multi-photo upload, cleanup/delete images.
7. **Alerts:** Web Push subscription, daily digest cron, in-app banner.
8. **Meal planning:** planner grid, `/api/suggest-meals`, mark-as-cooked, shopping list, saved recipes.
9. **Waste stats + polish:** Stats screen, undo toasts, empty states, dark mode, accessibility pass.
10. **Deploy:** Vercel, env vars, cron, test install on a real phone.

After each step: run type-check and lint, summarize what changed, and tell me how to test it before moving on.

## 12. Known Risks / Notes

- **iOS push** only works for PWAs added to the Home Screen (iOS 16.4+). Show an in-app prompt explaining this on first launch.
- **Web Speech API** is inconsistent on iOS Safari; keyboard dictation is the reliable fallback.
- **Receipt parsing** accuracy varies by store; the Review & Confirm screen is the safety net, so keep it fast.
- **AI expiry dates are estimates**; always label them and bias toward shorter shelf life for safety-critical items (meat, dairy, cooked food).
- Keep AI cost low: batch items per request, cache barcode lookups, use the shelf-life table before calling Claude.